PES UNIVERSITY • DEPT. OF COMPUTER SCIENCE AND ENGINEERING

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Student Name: Nagashri R Patil

SRN: PES1UG24CS289

Problem Statement: #43 – Feature Flag & Dynamic Config Manager

Course: Software Engineering Laboratory (Lab 2)

Agile Epics Specification

This document details the functional and architectural themes (Epics) derived from Lab 1 for **Problem Statement #43: Feature Flag & Dynamic Config Manager**. Each Epic represents a significant body of deliverable functionality encompassing user stories, business value, and technical constraints.

Epic 1: Flag Lifecycle & Metadata Management (FFM-EPIC-1)

Description: Enables software engineers to create, update, and manage feature flags with globally unique identifier keys, descriptions, default boolean states, and multi-environment overrides (Dev, Staging, Production). This prevents cross-environment configuration drift and provides a single source of truth for feature states.

Business Value: Decouples code deployment from feature release, enabling safe canary releases and dark launches without requiring code rollbacks.

Associated Requirements: FR-002, FR-004 | **Traceable Use Cases:** UC-01 (Create Feature Flag), UC-04 (Override Environment Config)

Epic 2: Targeting Rules & Gradual Rollouts (FFM-EPIC-2)

Description: Provides advanced rule evaluation mechanisms allowing engineers to segment users based on demographic attributes (e.g., beta cohort membership) and configure deterministic percentage-based rollouts (0–100%) calculated via a mathematical hash of the User ID.

Business Value: Limits blast radius during feature rollouts and empowers product teams to run safe A/B tests and early access programs.

Associated Requirements: FR-001, FR-003 | **Traceable Use Cases:** UC-02 (Configure Targeting Rules), UC-03 (Evaluate Feature Flag)

Epic 3: Governance, Approval Workflows & Audit Logging (FFM-EPIC-3)

Description: Establishes release governance by requiring a 4-eyes approval workflow where Release Managers inspect configuration diffs before approving changes to production. Maintains an append-only, tamper-proof audit log capturing author, approver, timestamp, and previous state with one-click rollback.

Business Value: Protects production stability against unauthorized or erroneous configuration shifts and ensures compliance with enterprise change control standards.

Associated Requirements: NFR-002 | **Traceable Use Cases:** UC-06 (Approve & Publish Flag Change), UC-07 (View Audit Log / Rollback)

Epic 4: High-Performance Evaluation & Real-Time SDK Sync (FFM-EPIC-4)

Description: Implements a sub-15ms p95 evaluation engine backed by in-memory indexing, complemented by a persistent WebSocket streaming architecture that pushes updated flag states to connected client SDKs in real time without requiring application restarts.

Business Value: Guarantees zero latency penalty on critical application paths and enables instantaneous global kill-switch capability during production incidents.

Associated Requirements: FR-005, NFR-001 | **Traceable Use Cases:** UC-05 (Sync Flags to Client SDK), UC-03 (Evaluate Feature Flag)

Epics Summary Table

Epic ID	Summary	Story Points	Scope	Sprint Allocation
FFM-EPIC-1	Flag Lifecycle & Metadata	8 PTS	Flag CRUD, Environment Overrides	Sprint 1 & Sprint 2
FFM-EPIC-2	Targeting & Percentage Rollout	10 PTS	Deterministic Hash, Attribute Rules	Sprint 1
FFM-EPIC-3	Governance & Audit Logging	13 PTS	Approval Gate, Diff Preview, Rollback	Sprint 2
FFM-EPIC-4	Evaluation Engine & SDK Sync	16 PTS	Sub-15ms p95, WebSocket Stream	Sprint 1 & Sprint 2

1. Epics Architecture

Epic ID	Summary	Scope	Traceability
FFM-EPIC-1	Flag Lifecycle & Metadata	Flag CRUD, metadata, multi-env overrides	FR-002, FR-004, UC-01, UC-04
FFM-EPIC-2	Targeting & Gradual Rollouts	Deterministic hashing, attribute targeting	FR-001, FR-003, UC-02, UC-03
FFM-EPIC-3	Governance & Audit Logging	Approval gate, visual diff, audit logging	UC-06, UC-07, NFR-002
FFM-EPIC-4	Evaluation Engine & SDK Sync	Sub-15ms p95, real-time WebSocket sync	FR-005, NFR-001, UC-05, UC-03

2. Product Backlog: User Stories Catalog

ID	Epic	User Story Statement	Priority	Pts	Sprint	Acceptance Criteria
US-01	FFM-EPIC-1	As a Software Engineer, I want to create a feature flag with a unique key, description, and default state, so that client apps can reference the flag unambiguously.	HIGH	3	Sprint 1	Pass: Duplicate key triggers 409 conflict; unique key registers < 2s. Fail: Duplicate keys accepted.
US-02	FFM-EPIC-2	As a Software Engineer, I want to configure a percentage rollout (0-100%) based on a deterministic hash of user IDs, so that I can gradually roll out features and limit blast radius.	HIGH	5	Sprint 1	Pass: 25% rollout allocates 25% (±2%) of 10,000 simulated IDs; hash mapping is strictly deterministic. Fail: User changes cohort across evaluations.
US-03	FFM-EPIC-2	As a Software Engineer, I want to evaluate user targeting rules for specific cohorts, so that targeted users receive early feature access.	HIGH	5	Sprint 1	Pass: User matching targeted attribute receives true; non-targeted user receives fallback. Fail: Non-targeted user receives feature.
US-04	FFM-EPIC-4	As a Software Engineer, I want the evaluation API to execute in under 15ms, so that embedding flag checks into critical paths adds negligible latency.	HIGH	8	Sprint 1	Pass: p95 latency under simulated peak load is < 15ms; in-memory cache lookup avoids disk I/O. Fail: Evaluation p95 exceeds 15ms.

ID	Epic	User Story Statement	Priority	Pts	Sprint	Acceptance Criteria
US-05	FFM-EPIC-4	As a Client SDK Developer, I want the SDK to maintain a local in-memory cache with fallback defaults, so that network partitions do not break host application features.	MEDIUM	3	Sprint 1	Pass: In-memory evaluations return in < 1ms during offline conditions; graceful fallback. Fail: Application blocks on network drop.
US-06	FFM-EPIC-1	As a Software Engineer, I want to define environment-specific overrides (Dev, Staging, Prod), so that I can test flags in lower environments without exposing code in Prod.	MEDIUM	5	Sprint 2	Pass: Prod override set to OFF while Staging is ON returns false for Prod and true for Staging. Fail: Overrides leak between environments.
US-07	FFM-EPIC-3	As a Software Engineer, I want to save changes as pending drafts and preview a visual diff, so that I can verify proposed modifications before requesting approval.	MEDIUM	3	Sprint 2	Pass: Draft changes do not alter runtime configurations; diff clearly highlights altered rules. Fail: Draft changes alter live evaluations.
US-08	FFM-EPIC-3	As a Release Manager, I want to review, approve, or reject pending flag changes with validation checks, so that invalid configs never reach production.	HIGH	5	Sprint 2	Pass: Only authenticated Release Managers can publish; invalid rules (rollout >100%) rejected. Fail: Unauthorized user publishes changes.
US-09	FFM-EPIC-3	As a Release Manager, I want all configuration changes logged in an immutable audit trail with one-click rollback, so that actions are auditable and regressions revert instantly.	HIGH	5	Sprint 2	Pass: Tamper-proof audit record created for every publish; one-click rollback restores prior state < 3s. Fail: Change occurs without audit entry.
US-10	FFM-EPIC-4	As a Client SDK Developer, I want updated flag states streamed via WebSockets in real time, so that clients update within 5 seconds without redeployment.	HIGH	5	Sprint 2	Pass: 95% of active SDK connections receive updates within 5s; automatic reconnect upon drop. Fail: Clients take > 30s to synchronize.

3. Planning Poker Estimation Rationale

Story points follow the Fibonacci sequence (`1, 2, 3, 5, 8, 13`), calibrating effort and risk:

- **3 Points:** Standard CRUD models and UI rendering (`US-01`, `US-05`, `US-07`).
- **5 Points:** Algorithmic hashing, multi-tenant state isolation, WebSocket channel management, and audit records (`US-02`, `US-03`, `US-06`, `US-08`, `US-09`, `US-10`).
- **8 Points:** Critical-path optimizations with high uncertainty (`US-04`: Sub-15ms p95 latency benchmark).

Sprint Burndown & Velocity Analytics

This report evaluates the execution metrics, velocity stability, and capacity dynamics observed across two simulated 1-week sprints for the Feature Flag & Dynamic Config Manager.

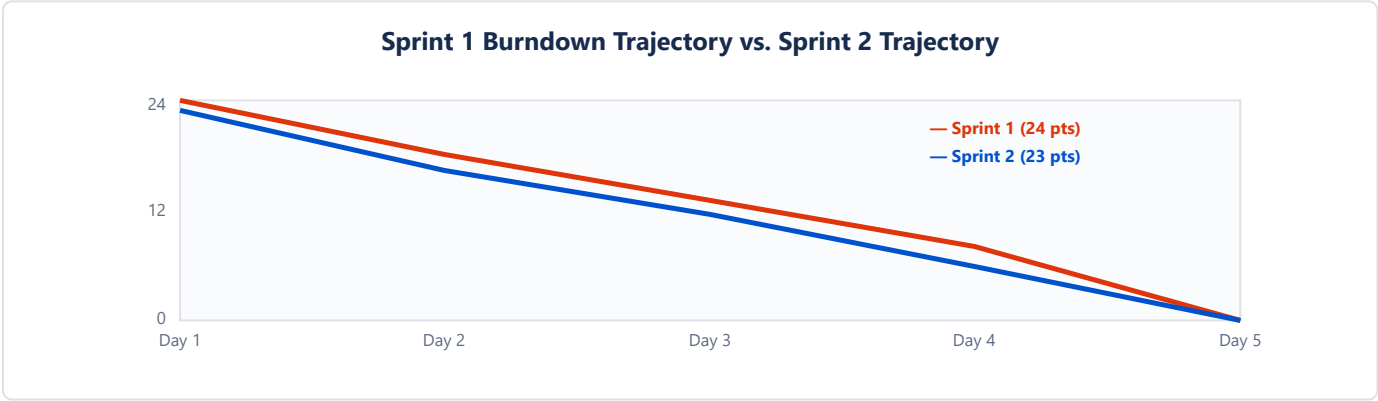
1. Sprint Execution Summary

Sprint Metric	Sprint 1 (Engine & Targeting)	Sprint 2 (Governance & Streaming)
Committed Points	24 Story Points (5 user stories)	23 Story Points (5 user stories)
Completed Points	24 Story Points (100% completion)	23 Story Points (100% completion)
Duration	1 week (5 working days)	1 week (5 working days)
Scope Creep	0 Story Points added	0 Story Points added
Pacing Trajectory	Step-like plateau on Day 3-4 (US-04 8pts)	Linear daily burndown (-5 to -8 pts/day)

2. Team Velocity & Capacity Insights

Sustainable Velocity: The team completed 24 points in Sprint 1 and 23 points in Sprint 2, establishing a highly stable velocity baseline of **~23.5 points per week**. This low variance (±1 point) gives project stakeholders confidence when forecasting subsequent product milestones.

Granularity & Story Sizing Impact: In Sprint 1, `US-04` was estimated at 8 points due to strict p95 < 15ms latency requirements. Because story points only burn upon achieving the "Definition of Done", the burndown chart exhibited a plateau between Day 2 and Day 4 while performance benchmarking took place. Once verified, points dropped sharply. Decomposing 8-point items into smaller 3-point slices in future sprints will yield a smoother, more predictable burndown trajectory.



3. Scope Discipline & Boundary Protection

Neither sprint experienced scope inflation (zero upward spikes). Feature requests identified mid-sprint (such as email notifications on flag approval) were logged into the product backlog for Sprint 3 planning rather than injected into running sprints.

Reflection Questions & Technical Analysis

1. Did your estimations reflect the actual effort?

Yes. Relative estimation using the Fibonacci sequence accurately captured development complexity, technical risk, and testing effort. Low-uncertainty CRUD items (e.g. US-01, 3 pts) progressed quickly without blockers. Algorithmic stories (e.g. US-02 deterministic hashing and US-09 cryptographic audit logging, 5 pts each) accounted for statistical cohort validation and transaction integrity testing. High-risk performance engineering (US-04 sub-15ms evaluation latency, 8 pts) correctly mirrored the iterative profiling, memory caching, and benchmark tuning needed to fulfill NFR-001. Abstracting effort into story points rather than calendar hours protected the team from false precision and Parkinson's Law.

2. Was your backlog well-prioritized?

Yes. Backlog prioritization followed architectural dependency sequencing and risk-reduction principles. Core flag creation (US-01) and deterministic evaluation (US-02, US-03) were scheduled at the top of Sprint 1 because downstream approval gates and WebSocket sync require a stable evaluation core to operate upon. Furthermore, scheduling the high-risk performance

optimization (US-04) in Sprint 1 eliminated architectural risk early. Enterprise governance, environment isolation, and real-time streaming were layered cleanly in Sprint 2, creating a resilient MVP progression.

3. How did your simulated sprint align with your plan?

The simulated sprints closely matched planned execution. In Sprint 1, 24 planned points were delivered across 5 user stories with zero scope creep. In Sprint 2, all 23 points were completed. Pacing was disciplined: smaller 3-point stories moved to Done early in both sprints, while complex 5-point and 8-point items progressed through comprehensive unit and load verification before transitioning to Done by Day 5. All sprint boundaries were protected against unplanned scope injections.

4. What insights did the burndown chart give about your team's capacity?

Jira's Burndown Chart revealed three key operational insights: (1) Predictable Capacity: Completing 24 points in Sprint 1 and 23 points in Sprint 2 established that the team has a reliable, sustainable velocity of ~23–24 points per 1-week sprint. (2) Story Sizing & Burndown Trajectory: Large work items (e.g. 8-point US-04) create step-like burndown plateaus because points only burn upon complete verification. Decomposing 8-point stories into smaller 3-point slices in future sprints will yield a smoother burndown curve. (3) Discipline & Zero Scope Creep: The absence of upward spikes on the remaining values line confirmed strict sprint backlog discipline without uncommitted task creep.